Classifying Self-Reported Heart Disease with NumPy: Feature Encoding and Decision Thresholds

Project 1 team

Abstract

We classify prevalent self-reported myocardial infarction or coronary heart disease in the 2015 BRFSS survey. Using only NumPy for modeling, we compare least squares, ridge regression, and regularized logistic regression with two feature representations. A stratified training/tuning/holdout protocol separates all model selection from evaluation. The selected model obtains holdout F1 0.4214 and accuracy 0.8736. We isolate the effects of feature encoding and threshold selection, exclude a target-dependent follow-up question, and provide a deterministic inference workflow.

1 Task and data

The supplied course dataset [1] contains 328,135 labeled respondents, 321 input variables, and 109,379 unlabeled test respondents. The positive class contains 28,975 respondents (8.83%). Labels are converted from $\{-1, +1\}$ to $\{0, 1\}$ for training and restored for submission. The target records whether respondents have ever reported myocardial infarction or coronary heart disease; it is derived from two diagnosis questions [2]. This is classification of reported disease history, not a prospective estimate of future heart attacks.

Class imbalance makes accuracy alone misleading: predicting the negative class achieves high accuracy while missing every positive. We therefore select using positive-class F1, $2TP/(2TP + FP + FN)$, consistent with the live leaderboard, and also report accuracy, precision, recall, and area under the ROC curve (AUROC). The competition's generic rules still name accuracy; reporting both avoids concealing this discrepancy [3].

2 Feature processing

All data-dependent transformations are fitted exclusively on the training partition. We exclude record identifiers `SEQNO`, `_PSU`, and `IDATE`; the CSV row identifier is never a predictor. We also explicitly exclude the target and its defining diagnosis questions if present. Crucially, we exclude `HAREHAB1`: it asks about rehabilitation following a heart attack, and its survey skip rule depends on a target-defining diagnosis [2]. Its values and missingness can therefore disclose the outcome.

Columns with more than 95% nonfinite entries are removed. Constant observed values are dropped, preserving a missingness indicator when applicable. Only nonfinite entries are considered missing: numbers such as 7 or 99 have variable-specific meanings and are not replaced globally. For each retained nonconstant column with missing entries, we append a binary missingness indicator.

The *numeric* representation imputes training medians, clips at the training 0.5th and 99.5th percentiles, then subtracts the training mean and divides by the training standard deviation. If both clipping quantiles coincide, the observed minimum and maximum are used instead. An intercept is included.

The *engineered* representation treats columns with at most 20 observed distinct values as categorical. Categories occurring fewer than 20 times share an "other" indicator with unseen categories; the most frequent category is the omitted reference. Missingness remains separate. Other columns receive the numeric transformation plus a separately standardized square, permitting nonlinear marginal effects without pairwise interactions. Both representations otherwise use the same selection and missingness rules. They yield 414 and 1136 development features, respectively. This ablation combines categorical encoding and squares, so it does not identify their separate contributions.

3 Models and evaluation protocol

For the predictive comparison, numeric least squares and ridge ($\lambda = 10^{-3}$) form linear baselines. Logistic regression is fitted on each representation with $\lambda \in \{10^{-5}, 10^{-4}, 10^{-3}\}$, minimizing

$$\frac{1}{n} \sum_i \Big[\log(1 + e^{x_i^T w}) - y_i x_i^T w \Big] + \lambda \|w_{1:}\|_2^2. \qquad (1)$$

The intercept is unpenalized in this additional predictive solver. A custom NumPy limited-memory BFGS optimizer uses ten history pairs, Armijo backtracking, gradient tolerance 10^{-6}, and at most 150 iterations. Stable sigmoid and log-sum-exp operations avoid exponential overflow. No class resampling or class weights are used.

We stratify by class into 60/20/20% training, tuning, and holdout sets with seed 433: 196,881, 65,627, and 65,627 rows. Each candidate's threshold is the tuning score maximizing F1, evaluating tied scores together. The candidate with the highest tuning F1 is frozen before computing any holdout metric. All fixed candidates are then evaluated on the same holdout for comparison; those results do not change the selection. This is one stratified split, not cross-validation.

Model	λ	Tune F1	Hold F1	Hold acc.
Majority	–	–	0.0000	0.9117
N Least squares	–	0.4213	0.4133	0.8621
N Ridge	10^{-3}	0.4201	0.4145	0.8603
N Logistic	10^{-5}	0.4260	0.4172	0.8624
N Logistic	10^{-4}	0.4260	0.4182	0.8596
N Logistic	10^{-3}	0.4269	0.4174	0.8770
E Logistic	10^{-5}	0.4313	0.4214	0.8736
E Logistic	10^{-4}	0.4312	0.4220	0.8699
E Logistic	10^{-3}	0.4285	0.4220	0.8697

Table 1: All fixed candidate results. N: numeric; E: engineered. Each nonconstant model uses its own tuning-selected threshold. Bold marks the model selected before holdout evaluation, not the largest holdout score.

Threshold	F1	Accuracy	Precision	Recall
0.5	0.2540	0.9142	0.5477	0.1653
0.21065	0.4214	0.8736	0.3536	0.5213

Table 2: Threshold ablation for the same selected model on the untouched holdout. The alternative threshold comes only from the tuning partition.

4 Results and ablations

Tuning selects engineered logistic with $\lambda = 10^{-5}$ and threshold 0.21065. Its holdout F1 is 0.4214, accuracy 0.8736, precision 0.3536, recall 0.5213, and AUROC 0.8578. The confusion counts are $TP = 3,021$, $FP = 5,522$, $FN = 2,774$, and $TN = 54,310$. The 95% percentile interval for F1 is [0.4113, 0.4319], from 2,000 multinomial resamples of these four counts (seed 434). This interval describes finite holdout sampling for a fixed fitted model; it excludes model-selection and training variability.

At the engineered model's tuning-selected $\lambda = 10^{-5}$, changing from numeric to engineered features changes holdout F1 by +0.0042. Independently tuning the regularization within each representation gives holdout F1 0.4174 (numeric) and 0.4214 (engineered). Thus Table 1 reports both a matched-regularization comparison and the complete small search, rather than selecting an ablation using holdout performance.

Replacing the default threshold 0.5 with the tuning-selected threshold changes holdout F1 by +0.1674 (Table 2). Lowering the threshold predicts more positives, trading precision and accuracy for recall. The threshold affects the classification decision, not the ranking or AUROC. The selected development optimizer used 150 iterations; reaching the iteration cap does not establish numerical convergence.

5 Reproducibility and limitations

After evaluation, we refit the frozen preprocessing family and model on all 328,135 labeled rows, retaining the tuning-selected threshold. The final representation has 1156 columns. The heldout estimates above refer to the development model, not this refit. No test labels or external data are used, and no online competition score is claimed.

The six required basic methods are provided in `implementations.py`, with gradient-based, stochastic, and normal-equation solvers returning final weights and unpenalized loss. `train.py` reproduces the split, candidate search, holdout evaluation, and refit; `run.py` loads saved JSON preprocessing and NumPy weights and generates the test CSV. Training saves split IDs, configuration, data hashes, metrics, and holdout predictions for auditing; per-row files are generated locally. The source repository includes all implementation details. Experiments used NumPy 2.5.4. This report is generated from `results/metrics.json`; preprocessing and thresholds are documented in code.

The experiment has a single split and a modest model search. The combined feature ablation cannot separate one-hot and quadratic effects, and optimizing a threshold on one tuning set can add selection variance. Treating refusal codes as values avoids invalid blanket replacements but does not exhaust the survey codebook's semantics. Survey design, self-report error, correlated variables, and distribution changes limit generalization; the resampling interval does not account for the complex survey design. These models support a reproducible course comparison, not clinical deployment or causal claims.

References

[1] EPFL CS-433. *Project 1 description*, Fall 2026. Course repository and supplied dataset.

[2] CDC. *2015 BRFSS Codebook Report*, pp. 73, 106, 2016. Official codebook.

[3] AIcrowd. *EPFL Machine Learning Project 1*. Leaderboard and rules, accessed 26 October 2026.